BIZEV-02: Instrumentation

> **Series:** BIZEV — Business Events & Funnel Analysis | **Notebook:** 2 of 7 | **Created:** March 2026 | **Last Updated:** 10/14/2026

Overview

Capturing meaningful business events requires deliberate instrumentation. This notebook covers the practical techniques for generating business events — from OneAgent capture rules on web request data, to custom API ingestion, the RUM and mobile `sendBizEvent` APIs, and OpenPipeline span-to-bizevent extraction. You will also learn best practices for event naming conventions, payload design, and cardinality management to keep your business analytics clean and performant.

Table of Contents

1. OneAgent Auto-Detection
2. Business Events API Ingestion
3. Code-Level Capture: RUM, Mobile and Workflows
4. OpenTelemetry Span-to-Bizevent Mapping
5. Event Naming Best Practices
6. Payload Design and Cardinality
7. Verifying Instrumentation with DQL
8. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	`storage:bizevents:read`, `bizevents.ingest`
OneAgent	Deployed on at least one application host (for auto-detection)
Knowledge	BIZEV-01 fundamentals, basic REST API concepts

> **No Grail yet?** Every capture path in this notebook writes to the Grail `bizevents` table and therefore requires a SaaS tenant with Grail. For what a classic (Gen2) environment can do instead — request attributes, session/action properties + USQL, log metrics — and for the hybrid pattern that adopts these capture paths with only a minimal Gen3 footprint, see **BIZEV-07: Gen2 vs Gen3 — Business Events Without (or Before) the Full Move**.

1. OneAgent Auto-Detection

OneAgent can automatically capture business events from web application requests using **capture rules**. This is the lowest-effort approach — no code changes required.

Setting Up Capture Rules

Navigate to **Settings > Collect and Capture > Business events** (**Incoming** or **Outgoing**) and define:

Setting	Description	Example
Trigger	URL pattern or request attribute match	`/api/v1/orders/*`
Event type	The `event.type` value to assign	`com.myapp.order.created`
Data extraction	Request/response attributes to capture	`order_id`, `total_amount`
Filtering	Conditions to limit capture	HTTP method == `POST`

What Gets Captured Automatically

When a capture rule matches, OneAgent creates a business event with:

- The configured `event.type`
- `event.provider` — the value you configure in the rule (for example, `www.easytrade.com`)
- Extracted request attributes as custom payload fields

> **Tip:** Start with auto-detection for existing web applications, then add custom instrumentation for backend processes that don't have HTTP endpoints.

```
// Check which business events your OneAgent capture rules are producing.
// A capture rule sets event.provider to the value you enter in the rule (for
example
// www.easytrade.com) — list the providers from your own rules here.
fetch bizevents, from:-24h
| filter in(event.provider, {"www.easytrade.com"})
| summarize event_count = count(), by:{event.type, event.provider}
| sort event_count desc
| limit 15
```

2. Business Events API Ingestion

The Business Events API allows you to send events from any system — backend services, batch processes, third-party integrations, or IoT devices.

CloudEvents Format

The API expects events in CloudEvents format:

```
{
  "specversion": "1.0",
  "id": "evt-20260312-001",
  "type": "com.myapp.payment.processed",
  "source": "payment-service",
  "time": "2026-03-12T10:30:00Z",
  "data": {
    "payment_id": "PAY-98765",
    "amount": 249.99,
    "currency": "USD",
    "method": "credit_card",
    "customer_tier": "gold"
  }
}
```

API Endpoint

```
POST https://{environment-id}.live.dynatrace.com/api/v2/bizevents/ingest
Content-Type: application/cloudevents+json
Authorization: Api-Token dt0c01.xxxxx
```

Batch Ingestion

For high-volume scenarios, send multiple events in a single request using `application/cloudevents-batch+json`:

```
[
  {"specversion": "1.0", "type": "com.myapp.event1", "source": "svc", "data":
{}},
  {"specversion": "1.0", "type": "com.myapp.event2", "source": "svc", "data":
{}}
]
```

> **Important:** The documented limit is on payload size — *"The Business events API limits payload size to 5 MB per request."* No request-rate figure is published, so for high throughput send batches under that size and retry with exponential backoff on `429` and `5xx` responses.
>
> **Sources:** Ingest business events via API (DT docs)

```
// Verify API-ingested events are arriving
// API-ingested events often have a custom source/provider value
fetch bizevents, from:-1h
| summarize {event_count = count(),
           latest = max(timestamp),
           earliest = min(timestamp)}, by:{event.provider}
| sort event_count desc
```

3. Code-Level Capture: RUM, Mobile and Workflows

The *Business event capture* page lists five capture methods — OneAgent capture rules, web and mobile RUM, external sources (the API), logs and spans (OpenPipeline), and Workflows. A OneAgent SDK is not among them, so there is no server-side SDK call to add to application code. Code-level capture is available on the client side, through the RUM JavaScript API and the mobile agents, and from automation, through Workflows. Backend code that has no capture rule sends events through the Business Events API (Section 2).

Browser (RUM JavaScript API)

```
dynatrace.sendBizEvent('com.myapp.add-to-cart', {
  product_id: 'PROD-456',
  product_name: 'Widget Pro',
  price: 29.99,
  quantity: 2
});
```

The method lives on the `dynatrace` object of the RUM JavaScript API, not on `dtrum`. Events are reported only for monitored sessions — if the RUM JavaScript is disabled for a session (for example by cost and traffic control), its business events are not sent.

Mobile and OpenKit

The mobile agents (Android, iOS, Cordova, Flutter, React Native, .NET MAUI, Xamarin) and OpenKit each provide their own business-event method. See the per-platform tabs on the *Get business events from web and mobile RUM* page for the exact call.

Workflows

To generate business events from automated tasks, add an **Ingest business event** action to a workflow.

> **Tip:** For backend services, prefer a OneAgent capture rule (Section 1) when the data is already in the request, or the Business Events API (Section 2) when it is not. Both avoid adding telemetry code to the application.

4. OpenTelemetry Span-to-Bizevent Mapping

If your application already produces spans with business-relevant attributes, OpenPipeline can create business events from them with a **Business event** processor in the **Data extraction** stage of the spans pipeline.

How It Works

1. Application emits spans with business attributes (e.g., `order.id`, `order.total`)
2. OpenPipeline receives the span data and a dynamic route sends it to your spans pipeline
3. A **Business event** processor in the **Data extraction** stage emits a **new** bizevent record for each matching span
4. The original span remains in `spans` — the business event is an additional record

OpenPipeline Configuration

In **Settings > Process and contextualize > OpenPipeline > Spans**, open the pipeline that processes the spans, then go to **Data extraction** and add a **Business event** processor:

Setting	Value
Matching condition	`span.name == "checkout.complete"`
Event type (static string)	`com.myapp.checkout.completed`
Event provider (static string)	`checkout-service`

Setting	Value
Field extraction	`order.id`, `order.total` (or *Extract all fields*)

OpenPipeline matchers accept **double-quoted** strings only; a single-quoted value such as `'checkout.complete'` is rejected. Finish by adding a **Dynamic routing** entry that sends the relevant spans to this pipeline.

> **Note:** This approach avoids double-instrumentation. If your spans already carry business data, extract it in OpenPipeline rather than adding a second telemetry call to the code.

```
// Which trace-correlation fields do your business events actually carry?
// The bizevents model in the semantic dictionary lists none, and the field
names differ by
// capture path — so discover them on your own records before filtering on any
of them.
// Wildcard fieldsKeep returns only the matching fields that exist.
fetch bizevents, from:-24h
| fieldsKeep event.type, event.provider, "*trace*", "*span*"
| limit 20
```

5. Event Naming Best Practices

A consistent naming convention is critical for long-term analytics. Poor naming leads to fragmented data and unreliable queries.

Recommended Naming Convention

Use reverse-domain notation with a verb suffix:

```
com.&lt;company&gt;.&lt;domain&gt;.&lt;action&gt;
```

Pattern	Example	Description
`com.myapp.order.created`	Order placed	New order in system
`com.myapp.order.completed`	Order fulfilled	Order shipped/delivered

Pattern	Example	Description
`com.myapp.payment.processed`	Payment received	Successful payment
`com.myapp.payment.failed`	Payment rejected	Failed payment attempt
`com.myapp.user.signup`	New registration	User account created
`com.myapp.user.login`	Authentication	Successful login
`com.myapp.cart.updated`	Cart change	Item added/removed

Naming Anti-Patterns

Avoid	Why	Better
`order`	Too vague — created? cancelled?	`com.myapp.order.created`
`OrderCreated`	Inconsistent casing	`com.myapp.order.created`
`com.myapp.order.created.v2`	Version in name fragments data	Use payload versioning
`event_12345`	Meaningless identifier	Use descriptive domain names

```
// Audit event naming — find event types that may need standardization
fetch bizevents, from:-7d
| summarize {event_count = count(),
           first_seen = min(timestamp),
           last_seen = max(timestamp)}, by:{event.type}
| sort event_count desc
```

6. Payload Design and Cardinality

The custom attributes you include in business events determine what analytics are possible. Design payloads carefully.

Payload Design Principles

Principle	Guidance
Include business identifiers	`order_id`, `customer_id`, `session_id`
Include measurable values	`amount`, `quantity`, `duration_ms`
Include categorical dimensions	`category`, `region`, `customer_tier`
Avoid high-cardinality strings	Don't include full URLs, stack traces, or free text
Use consistent types	Always send `amount` as a number, not sometimes as a string

Cardinality Management

High-cardinality fields (fields with millions of unique values) can degrade query performance.

Field Type	Cardinality	Impact
`customer_tier` (gold/silver/bronze)	Low (~3 values)	Excellent for `summarize by:`
`product_category`	Medium (~100 values)	Good for grouping
`order_id`	High (millions)	Fine for lookup, avoid `summarize by:`
`full_url`	Very high	Avoid — parse into structured fields instead

> **Warning:** Using `summarize by:{order_id}` on millions of unique order IDs will produce a massive result set and slow queries. Use high-cardinality fields for filtering (`filter order_id == "ORD-123"`) rather than grouping.

```
// Assess cardinality of common fields in your business events
fetch bizevents, from:-24h
| summarize {total = count(),
           distinct_types = countDistinct(event.type),
           distinct_providers = countDistinct(event.provider),
           distinct_categories = countDistinct(event.category)}
```

7. Verifying Instrumentation with DQL

After setting up instrumentation, validate that events are arriving correctly and contain the expected data.

```
// Check for gaps in business event ingestion over the last 24 hours
// A healthy pipeline should show consistent event flow
fetch bizevents, from:-24h
| makeTimeseries event_count = count(), interval:15m
```

```
// Validate that required fields are populated (not null)
fetch bizevents, from:-1h
| summarize {total = count(),
           has_type = countIf(isNotNull(event.type)),
           has_provider = countIf(isNotNull(event.provider)),
           has_category = countIf(isNotNull(event.category))}
| fieldsAdd type_pct = round(toDouble(has_type) / toDouble(total) * 100,
decimals: 1),
           provider_pct = round(toDouble(has_provider) / toDouble(total) *
100, decimals: 1),
           category_pct = round(toDouble(has_category) / toDouble(total) *
100, decimals: 1)
```

8. Summary and Next Steps

In this notebook, you learned:

- **OneAgent capture rules** — Capture business events from web requests without code changes
- **API ingestion** — Send events from any system using CloudEvents format
- **Code-level capture** — `dynatrace.sendBizEvent` (RUM JavaScript), the mobile agents, and the Workflows *Ingest business event* action

- **Span extraction** — A Business event processor in the OpenPipeline Data extraction stage creates bizevents from spans
- **Naming conventions** — Use reverse-domain notation with action verbs
- **Payload design** — Balance richness with cardinality management

Next Steps

- **BIZEV-03: Funnel Analysis** — Use your instrumented events to build conversion funnels
- **BIZEV-05: KPIs and Metrics** — Extract business KPIs from event data

References

- Environment API (DT docs)
- Get business events via OneAgent (DT docs) — *"OneAgent Full-Stack Monitoring mode is mandatory for the hosts in which you want to capture business events."*
- Get business events from web and mobile RUM (DT docs) — *"Business events are available for all Dynatrace RUM technologies (web RUM, mobile RUM, and OpenKit)."*
- RUM JavaScript API — dynatrace (DT docs) — *"Business events are only supported on Dynatrace SaaS deployments currently."*
- Business event capture (DT docs) — *"Use the Ingest business event action within Workflows to generate business events from automated tasks."*
- Get business events from logs and spans (DT docs) — *"When spans are captured by OneAgent, span-level request attributes are available and can be mapped as data fields."*
- DQL matcher in OpenPipeline (DT docs)
- OpenPipeline Processing

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.